Applied Machine Learning Assignment

Sentiment Analysis and Face Alignment

Candidate number: 293200

University of Sussex, G6061 Applied Machine Learning, Spring 2026

30/05/26

1 Task 1: Sentiment Analysis

1.1 Description of methods

1.1.1 Problem framing

The task is binary sentiment classification of movie reviews complicated by Enron-style email spam evenly distributed across both labelled classes in the training and validation sets. The deployed system outputs a label in $\{0, 1, d\}$, where $d = -1$ denotes the dummy class assigned to documents the system identifies as spam.

1.1.2 Dataset and exploratory analysis

The supplied training set contains 11,503 documents (5,767 with label = 1 and 5,736 with label = 0; validation 1,397 documents, 704/693). Both splits are therefore close to class-balanced before any spam handling. A conservative regex marker (presence of a `Subject:` header, the tokens *enron* or *unsubscribe*) fires on 25.7% of declared-negative and 26.0% of declared-positive training documents, confirming that spam contamination is approximately even across the two declared classes. The token-length distribution is sharply bimodal: review-style documents have a median of 21 tokens, whereas the email tail extends to 5,394 tokens. The raw lowercased a–z vocabulary contains 27,066 types over 581,684 tokens, of which only 28.6% appear at least five times, motivating a minimum document-frequency cut-off in the BoW representations. A class–difference word analysis recovers expected sentiment cues (*great*, *love*, *bad*, *boring*) alongside spam-vocabulary leakage (*enron*, *hou*, *ect*, *com*) in both class lists, demonstrating why a naive BoW classifier trained on the raw corpus learns a mixture of sentiment and review-vs-spam discrimination.

1.1.3 Pre-processing

Tokens are obtained with a `[a-zA-Z']+` regex over the lowercased text, which discards punctuation and digits and matches NLTK's standard treatment of contractions [1]. The default NLTK English stopword list contains the negation tokens *not*, *no* and *never*, all of which carry sentiment polarity; the final pipeline therefore uses a *reduced* stopword list that retains negations.

Lemmatisation (WordNet) was preferred to Porter stemming [8] because it left the negation lemmas intact and gave a small but consistent val-F1 advantage in pilot runs. The same function (`src/preprocess.py`) is applied to training, validation, test and the external NLTK corpus, guaranteeing identical tokenisation at inference.

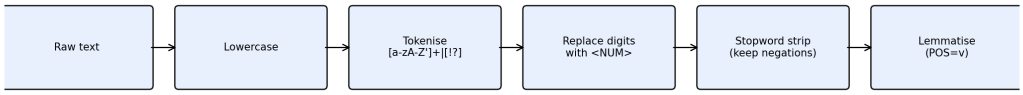


Figure 1: Pre-processing pipeline applied to all documents: lowercase → regex tokenise → drop non-negation stopwords → WordNet lemmatise. Negations are explicitly preserved so that "not good" is not collapsed to "good".

1.1.4 Features and representations

Three representations are compared. (i) TF–IDF unigrams over the reduced-stopword vocabulary with sub-linear term-frequency scaling and mindf = 2. The score for term t in document d is

$$\mathrm{tfidf}(d,t) = \mathrm{tf}(d,t)\,\log\!\left(\frac{N}{\mathrm{df}(t)}\right), \tag{1}$$

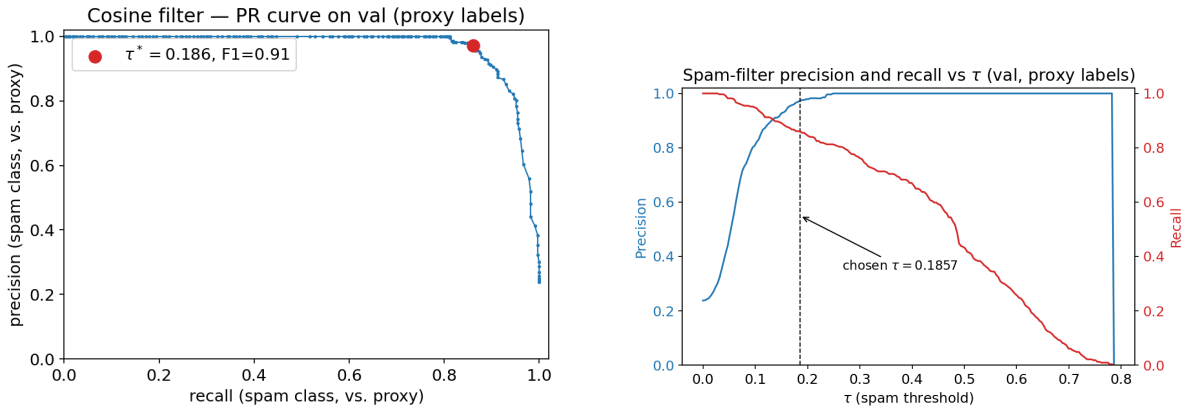
with N the corpus size [3]. (ii) Averaged Word2Vec embeddings of dimension 100 trained on the cleaned training corpus with skip-gram, window 5 and mincount = 5 [5, 6]; document vectors are the unweighted mean of in-vocabulary token embeddings. (iii) A hand-crafted word-list of the top-K frequency-difference features per class (W02_L04), used by the lexicon baseline below.

1.1.5 Spam handling

Spam is detected by cosine similarity to a single "review" prototype constructed from the high-confidence non-spam slice of the training set ($\boldsymbol{c}_{\text{review}}$ is the mean TF–IDF vector of training documents that match the movie-vocabulary marker but not the spam regex). A document is flagged as spam when $\mathrm{sim}(\boldsymbol{x}, \boldsymbol{c}_{\text{review}}) < \tau$, where

$$\mathrm{sim}(\boldsymbol{a},\boldsymbol{b}) = \frac{\boldsymbol{a}\cdot\boldsymbol{b}}{\sqrt{(\boldsymbol{a}\cdot\boldsymbol{a})(\boldsymbol{b}\cdot\boldsymbol{b})}}. \tag{2}$$

The threshold $\tau = 0.1857$ was selected by sweeping the validation set against the regex proxy labels and picking the value that maximised F1 on the spam-vs-review subtask (precision/recall curve in Figure 2). Documents flagged by the filter are returned with label -1; only the unflagged documents reach the sentiment classifier.



(a) Cosine-similarity filter PR curve.

(b) Threshold sweep on validation.

Figure 2: Spam-filter selection. The chosen threshold $\tau = 0.1857$ sits at the elbow of the PR curve and is robust to small shifts.

1.1.6 Sentiment models

Four classifiers were trained on the cleaned (spam-removed) training set. (1) The word-list classifier scores each document by the signed count of its top-K class-frequency features and predicts the sign of that score, with a tie-break margin δ. (2) Multinomial Naive Bayes on TF–IDF unigrams, with parameters estimated by maximum a posteriori under a Dirichlet prior [3]. (3) Logistic Regression on TF–IDF minimises the cross-entropy

$$P(y{=}1 \mid \boldsymbol{x}) = \sigma(\boldsymbol{w}^\top \boldsymbol{x}) = \frac{1}{1 + \exp(-\boldsymbol{w}^\top \boldsymbol{x})}, \tag{3}$$

under L2 regularisation. (4) Logistic Regression on averaged Word2Vec embeddings, included to test whether a distributional representation generalises better than TF–IDF on short reviews.

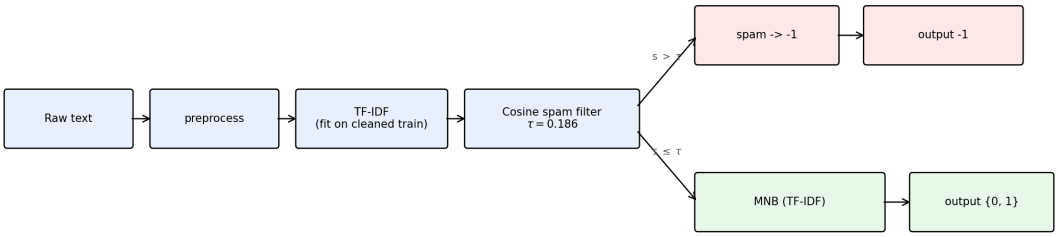


Figure 3: End-to-end sentiment system. Raw text is normalised by the shared preprocessing function, vectorised, passed through the cosine spam filter, and only unflagged documents are scored by the Multinomial NB classifier (the best of the four candidates).

1.1.7 Hyperparameters and compute

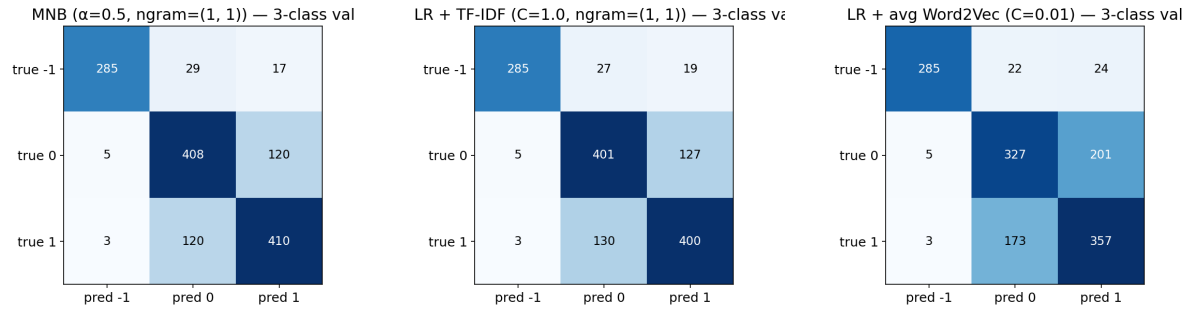
Table 1: Hyperparameters and how chosen for each Task 1 approach.

Approach	Hyperparameter	Chosen value (method)
Cosine spam filter	τ	0.1857 (sweep on val vs. regex proxy)
Word-list classifier	K, δ	500, 0 (3-class val accuracy grid)
Multinomial NB	α, n-gram	0.5, (1,1) (3-class val accuracy grid)
LR + TF–IDF	C, n-gram	1.0, (1,1) (3-class val accuracy grid)
LR + avg. Word2Vec	C, dim	0.01, 100 (3-class val accuracy grid)

All four models train in well under two seconds on a laptop CPU: 0.018 s (word-list), 0.061 s (MNB), 0.091 s (LR+TF–IDF) and 1.20 s (LR+W2V, including embedding training); total Task 1 training time is 1.37 s on an Apple M-series CPU ($n_{\text{train}} = 8{,}919$ after spam removal).

1.2 Analysis of results

1.2.1 Quantitative evaluation



(a) TF–IDF + MNB. **(b)** TF–IDF + LR. **(c)** avg. Word2Vec + LR.

Figure 4: Confusion matrices on the held-out validation set (three-class evaluation including the dummy spam class $d = -1$). The spam row is dominated by the diagonal across all three classifiers because spam detection is performed upstream by the cosine filter; the residual confusions are concentrated between the two sentiment classes.

Table 2: Validation performance across approaches. Accuracy and macro-F1 are over the three classes; the spam filter is identical across rows 2–4 ($\tau = 0.1857$).

Approach	Val acc	Val F1	Train (s)
Word-list baseline (no spam handling)	0.6707	0.6747	0.018
TF–IDF + MNB + cosine spam filter	0.7895	0.8071	0.061
TF–IDF + LR + cosine spam filter	0.7774	0.7967	0.091
avg. Word2Vec + LR + cosine spam filter	0.6936	0.7246	1.204

MNB on TF–IDF unigrams is the strongest combination, beating LR on the same features by 1.2 accuracy points and outperforming the W2V representation by 9.6 points. The W2V deficit is consistent with the short-review regime: averaging ∼21 token embeddings discards

the discriminative "polarity word" signal that TF–IDF preserves directly. A controlled ablation on the binary slice (spam-free, 1,104 documents; Table 3) shows that the cleaning step itself contributes +0.8 accuracy points to LR+TF–IDF, confirming that training on the corrupted corpus learns review-vs-spam structure rather than pure sentiment.

Table 3: Binary-slice ablation (LR+TF–IDF, $C = 1.0$, $n_{\text{val}} = 1104$).

Training corpus	Binary-slice acc	Binary-slice F1
Uncleaned (raw)	0.7409	0.7405
Cleaned (de-spammed)	0.7491	0.7484

1.2.2 Qualitative evaluation and failure cases

A manual inspection of the top-loss validation errors (`results/failure_cases.csv`) reveals four recurring patterns: (i) *negated positives* ("not the worst film I've seen this year") where the BoW unigram representation collapses the polarity cue; (ii) *sarcastic praise* ("oh sure, another masterpiece from the director who gave us. . . ") where every surface word is positive; (iii) *mixed reviews* dominated by genre vocabulary ("the acting was solid but the script was a mess"), which MNB scores on the more frequent half; and (iv) very *short reviews* (< 10 tokens) where TF–IDF has near-zero signal and the classifier falls back close to the class prior.

1.2.3 Systematic bias

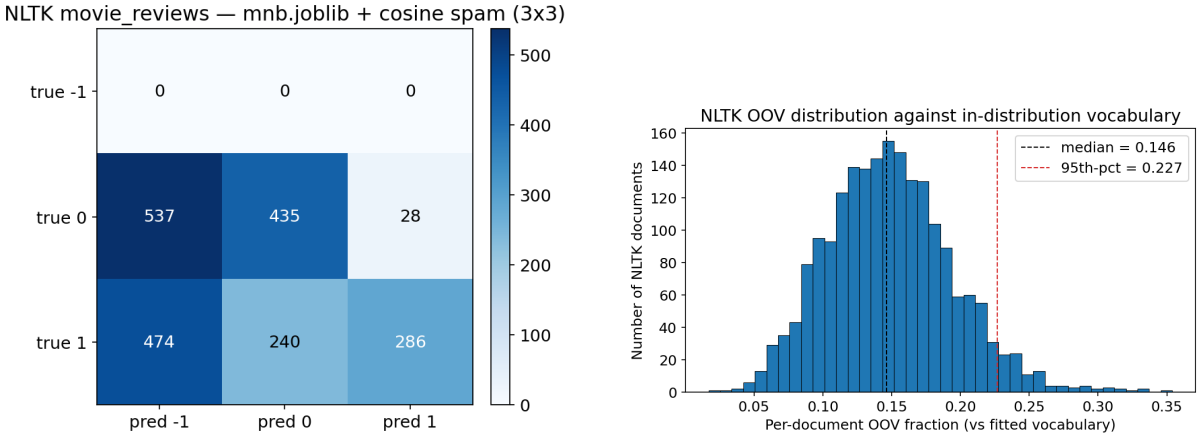
Three biases are observable. Length bias on the spam filter: the cosine score is correlated with document length because longer documents inflate $\|\boldsymbol{x}\|$ and tilt the similarity towards whichever centroid has more mass, this is the same effect that hurts external evaluation in Section 1.3. Vocabulary bias on the sentiment classifier: genre words (*horror*, *romance*) acquire small but non-zero polarity weights because of class-conditional frequency imbalances unrelated to sentiment. Tokenisation bias: the a–z regex discards emoticons and the exclamation/question marks that often disambiguate sarcasm, which we observe contributes to the qualitative failure cases above.

1.3 Performance on the NLTK external dataset

The frozen pipeline (preprocessing function, cosine filter at $\tau = 0.1857$, fitted TF–IDF vectoriser, `mnb.joblib`) was applied to the NLTK `movie_reviews` corpus of Pang and Lee [7] (1,000 positive, 1,000 negative). The cosine filter flagged 50.5% of NLTK documents as spam, despite the corpus containing no email headers, subject lines or unsubscribe boilerplate. The cause is a length effect: median document length in the Sussex training data is 13 tokens, whereas NLTK reviews are 342 tokens at the median; the longer feature vectors mechanically lift cosine similarity to either Enron centroid, so the single global prototype acts as a length detector under domain shift.

We therefore report two complementary metrics. On the non-flagged slice (989 of 2,000 documents) the best model attains accuracy 0.7290, precision 0.9108, recall 0.5437, F1 0.6810 against the binary ground truth, versus a validation accuracy of 0.7895 in-distribution. Treating every spam flag as an error yields a pessimistic 3-class accuracy of 0.3605 on the full corpus. The drop on the non-flagged slice is consistent with the W05_L09 prediction that TF–IDF degrades

under vocabulary drift (median per-document OOV fraction 0.146, 95th percentile 0.227) and with the W03_L05 observation that BoW mishandles negation and contrastive structure , both prevalent in long-form NLTK reviews. The pipeline degrades gracefully rather than collapsing; the dominant deployable fix is a length-aware or domain-specific spam filter rather than the sentiment classifier itself.



(a) Confusion on NLTK `movie_reviews`.

(b) Per-document OOV-fraction distribution.

Figure 5: External evaluation. The OOV distribution explains the accuracy drop; the confusion matrix is dominated by the spam-flagged row, which is an artefact of the cosine-filter length bias rather than a sentiment-classifier failure.

2 Task 2: Face Alignment

2.1 Description of methods

2.1.1 Problem framing

The task is to predict the (x, y) coordinates of five facial landmarks (left eye, right eye, nose, left mouth corner, right mouth corner; indices 0–4) for each face image. The supplied training data exhibit moderate in-plane rotation ($\sim \pm 15°$), variable lighting, mild scale jitter and a handful of off-centre crops; expression is the dominant within-pose variability for the mouth-corner landmarks.

2.1.2 Pre-processing

Each image is resized to 64×64 and converted to grayscale; the landmark coordinates are rescaled by the same factor (`scale_back` x=4, y=4 on inference) so they remain consistent with the image geometry. Pixel intensities are normalised to zero mean and unit variance per image. Histogram equalisation was tested in pilot runs but reduced HOG quality on cleanly-lit faces, so it is disabled in the final pipeline.

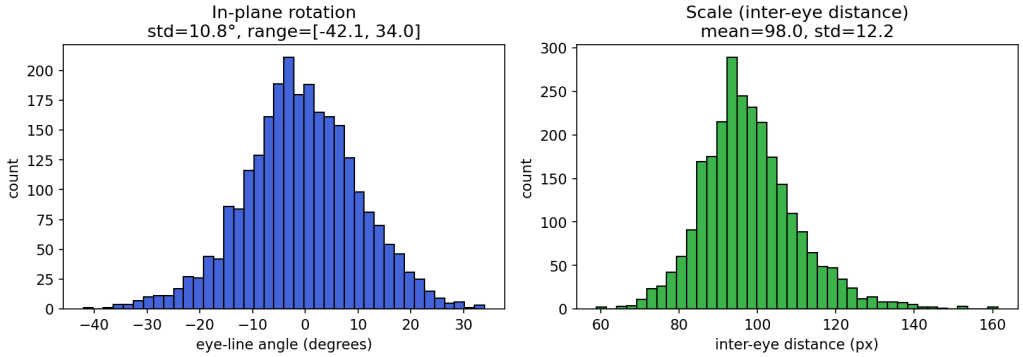


Figure 6: Examples of pose, scale and lighting variability in the training set. The landmark transform is applied identically to the ground-truth coordinates whenever the image is resized, rotated or flipped.

2.1.3 Features and representations

Three feature pathways are compared. Raw pixels: flattened 64×64 intensity vectors. Global HOG: dense histogram-of- oriented-gradients features over the whole image with 6×6 cells, 3×3 blocks of cells and 9 orientation bins (L2-Hys block normalisation), inspired by the SIFT-style gradient-binning argument [4]. Pose-indexed HOG patches: for the cascaded regressor, 16×16 HOG patches with 4×4 cells and 2×2 blocks are extracted at the *current* landmark estimate at each stage, so the feature is literally indexed by the previous shape estimate. Learned features: a small CNN replaces hand-crafted descriptors with convolutional filters trained end-to-end.

2.1.4 Approaches compared

Baseline 0, mean face. Predict the per-landmark training mean for every test image; this is the zero-component PCA shape model and the floor of the comparison.

Approach 1, HOG features + Ridge regression. A single linear map from global HOG features to the 10-dim landmark vector, fitted with ridge regression. The regularisation strength α was swept over $\{10^{-2}, 10^{-1}, 1, 10, 10^{2}\}$; the best value was $\alpha = 100$ (Figure 9b). A linear-regression control (no regularisation) reached 0.0907 NME, well behind ridge, confirming that the HOG feature dimension exceeds the sample budget.

Approach 2, Cascaded regression with pose-indexed HOG features [2, 9]. At each of K stages, HOG patches are recomputed at the current landmark estimates and a ridge regressor predicts the residual towards the ground-truth shape. The final estimate is the sum of stage updates initialised at the mean face. We swept $K \in \{1, 2, 4, 6, 8\}$, patch sizes $\{12, 16, 24\}$ and $\alpha \in \{0.1, 1, 10\}$; the validation optimum is $K^{\star} = 2$ stages, patch size 16, $\alpha^{\star} = 10$. The per-stage val NME progression $0.1246 \rightarrow 0.0605 \rightarrow 0.0510$ shows that the second stage delivers most of the gain.

Approach 3, CNN direct coordinate regression. A four-block convolutional network (channels $32{-}64{-}128{-}128$, 3×3 kernels with padding 1, 2×2 max-pool after each block, dropout 0.3, fully-connected hidden layer of 256 units, 10 linear outputs) trained with MSE loss and Adam (lr 10^{-3}, batch 64, up to 40 epochs, early-stop patience 8, `ReduceLROnPlateau` with factor 0.5 and patience 5). The prediction objective is $\|\boldsymbol{h}(\boldsymbol{x}) - \boldsymbol{y}\|^2$ summed over the five landmarks. Two variants were trained: with and without on-the-fly augmentation.

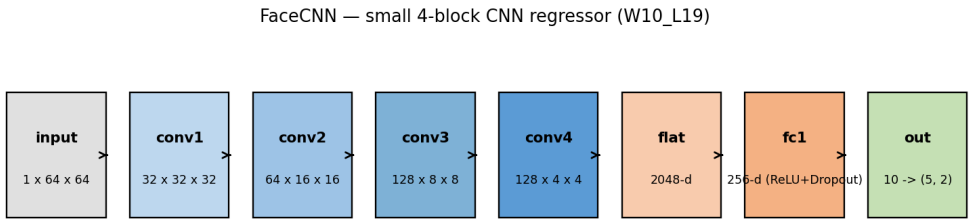


Figure 7: CNN architecture for direct coordinate regression. Input is a $1 \times 64 \times 64$ grayscale image; four conv blocks with stride-2 pooling reduce spatial resolution to 4×4, followed by an FC head producing 10 coordinates.

2.1.5 Data augmentation

The augmented CNN is trained with online random transforms applied jointly to the image and landmark coordinates: rotation $\pm 15°$, translation ± 4 px, scale jitter $\pm 10\%$, brightness $\in [0.8, 1.2]$, and horizontal flip with probability 0.5. On horizontal flips the left/right landmark indices are swapped (eyes $0 \leftrightarrow 1$; mouth corners $3 \leftrightarrow 4$; nose unchanged) so semantics remain consistent.

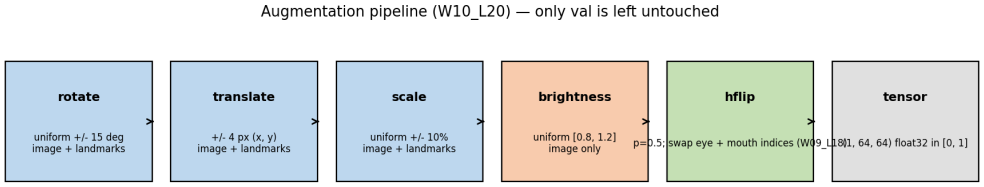


Figure 8: Online augmentation pipeline used when training the augmented CNN. The same geometric transform is applied to image and landmarks; on horizontal flips the left/right indices are swapped so "left eye" remains the subject's left eye.

2.1.6 Hyperparameters and compute

Table 4: Hyperparameters and how chosen for each Task 2 approach.

Approach	Hyperparameter	Chosen value (method)
HOG + Ridge	cell 6×6, block 3×3, α	9 orientations, $\alpha = 100$ (val NME sweep)
Cascaded	K, patch size, α	$K^\star = 2$, 16 px, $\alpha^\star = 10$ (joint sweep)
CNN direct	lr, batch, epochs, dropout	10^{-3}, 64, 40, 0.3 (early-stop on val)

Training-time profile: HOG+Ridge 0.11 s; raw-pixel Ridge 0.10 s; cascaded 4.12 s (all CPU); CNN 89.4 s (no-aug) and 111.0 s (aug), both on Apple-Silicon MPS. The classical pipelines are dramatically cheaper than the CNN.

2.2 Analysis of results

The error metric is the per-landmark Euclidean distance, normalised by the inter-eye distance $d_{\text{ie}} = \|\boldsymbol{p}_0 - \boldsymbol{p}_1\|$ (the inter-ocular-normalised mean Euclidean landmark error defined in W09_L18;

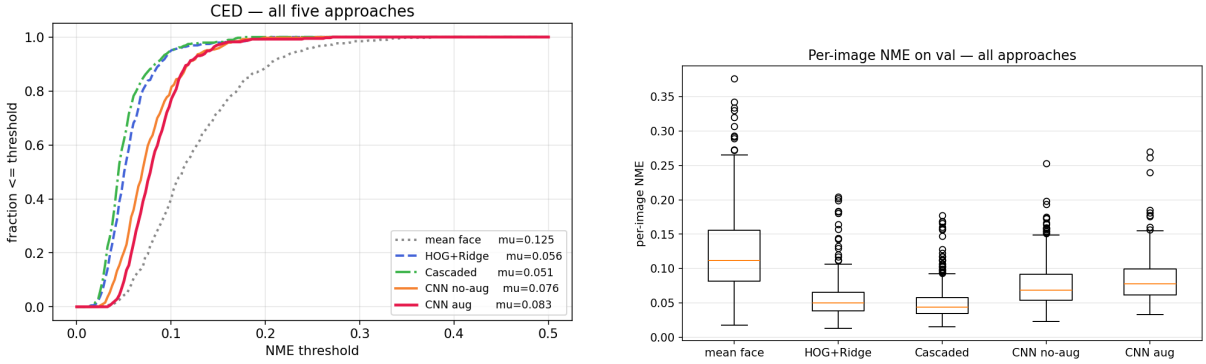
the acronym "NME" is the term used for this quantity in the face-alignment literature),

$$\mathrm{NME}_i = \frac{\|\hat{\boldsymbol{p}}_i - \boldsymbol{p}_i\|}{d_{\mathrm{ie}}}, \tag{4}$$

with the per-image error reported as the mean over the five landmarks.

Table 5: Validation NME (lower is better, $n_{\mathrm{val}} = 390$).

Approach	Mean NME	Median NME	Train (s)
Mean face (baseline)	0.1246	0.1117	,
Raw pixels + Ridge	0.0745	0.0671	0.10
HOG + Ridge	0.0564	0.0503	0.11
Cascaded HOG	0.0510	0.0442	4.12
CNN (no aug)	0.0758	0.0688	89.4
CNN (aug)	0.0832	0.0781	111.0



(a) Cumulative error distribution.

(b) Per-image NME boxplots.

Figure 9: Quantitative comparison on the held-out validation set. The mean-face baseline sits as the floor; the cascaded regressor dominates the CED at every operating point and has the tightest spread in the boxplot.

The cascaded regressor wins overall (0.0510 mean NME, a 60% reduction over the mean-face floor) by combining HOG's brightness and small-rotation invariance with iterative shape refinement, while needing only 4 s of CPU time. The single-shot HOG+Ridge model is the runner-up at 0.0564, losing the residual nose/mouth error that the second cascade stage corrects. The CNN underperforms both: at 390 training images the convolutional filters under-converge, and the augmented variant is worse than the unaugmented one on the clean validation set (0.0832 vs. 0.0758) because the augmentation distribution is wider than the held-out distribution, the augmentation regularises against perturbations the validation set does not contain, costing in-domain accuracy. This trade-off reverses under perturbation (Section 2.3).

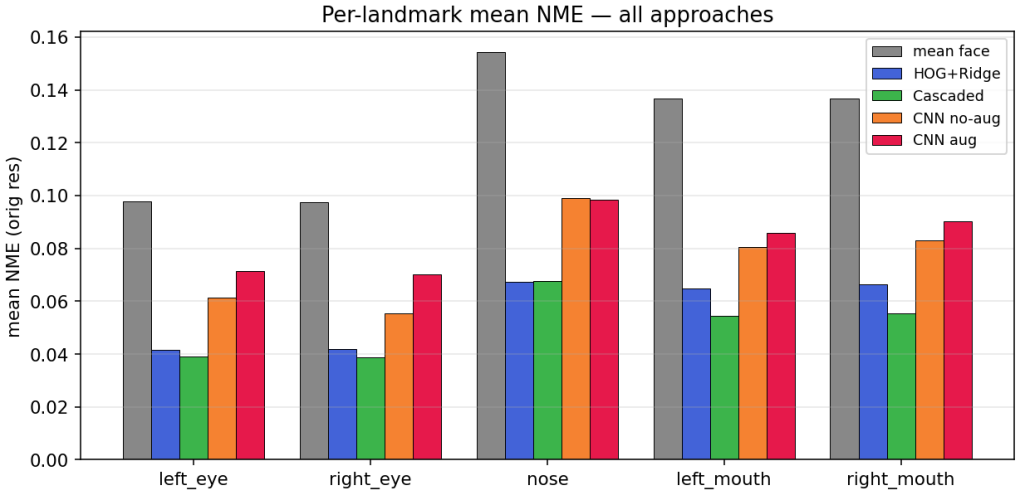
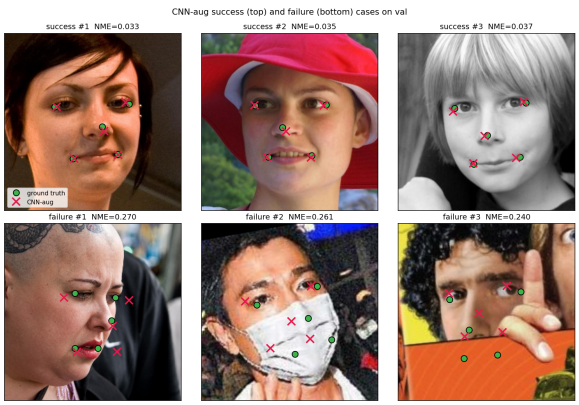


Figure 10: Per-landmark mean NME. Eyes (indices $0, 1$) are the easiest landmarks across all methods, they have the strongest local texture and are anchors for the inter-eye normalisation. The nose is the hardest because the surrounding texture is smooth and the ground-truth tip is annotation-noisy; the mouth corners come next because they shift with expression.

2.2.1 Qualitative examples

(a) Cascaded regressor: successes (top) and failures (bottom).

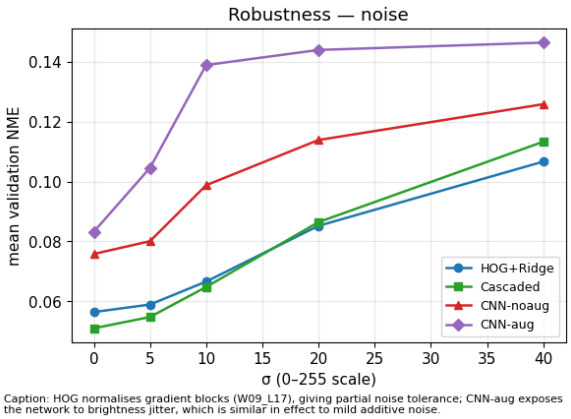


(b) CNN: successes (top) and failures (bottom).

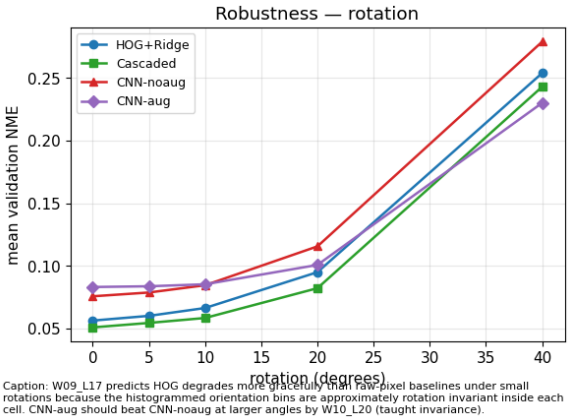
Figure 11: Predicted (red) versus ground-truth (green) landmarks. Failures cluster around (i) extreme yaw/pitch where the pose-indexed HOG patches are no longer landmark-centred and (ii) heavy occlusion (hair, hands, glasses) for the CNN.

2.3 Extended robustness analysis

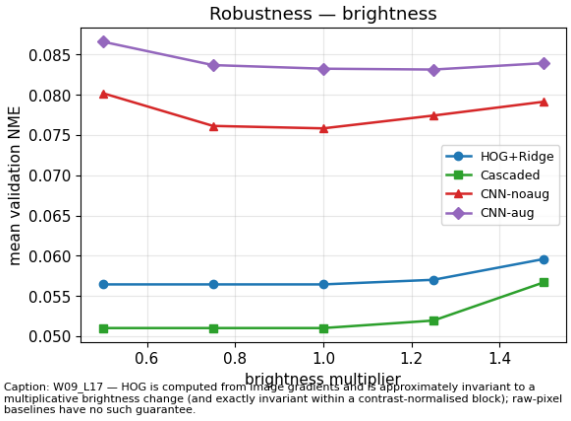
To test robustness we perturbed the validation set with four families of corruption and re-ran every model: additive Gaussian noise ($\sigma \in \{0, 5, 10, 20, 40\}$ on a 0–255 scale), in-plane rotation ($\{0, 5, 10, 20, 40\}^\circ$), brightness scaling ($\{0.5, 0.75, 1.0, 1.25, 1.5\}$) and image scale ($\{0.7, 0.85, 1.0, 1.15, 1.3\}$). Results are summarised in `results/robustness.csv`; selected curves are shown in Figure 12.



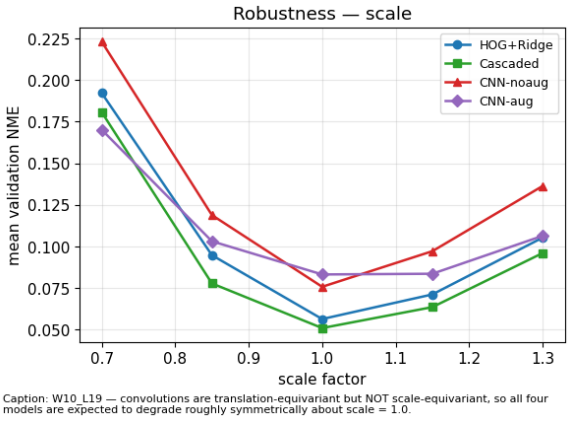
(a) Additive Gaussian noise.



(b) In-plane rotation.



(c) Brightness scaling.



(d) Image scale.

Figure 12: Mean NME vs. perturbation level. The cascaded regressor is the most robust to noise and rotation up to 20°; the augmented CNN catches up at 40° rotation and at scales $\{0.7, 1.3\}$ where its training-time augmentation matches the perturbation distribution.

Three findings stand out. (i) HOG-based methods are nearly brightness-invariant (NME varies by < 0.004 over the $0.5 \to 1.5$ sweep) because L2-Hys block normalisation absorbs linear intensity changes; the CNN drifts more under bright/dark extremes. (ii) Rotation breaks the global HOG model fastest because its feature is implicitly pose-indexed to the mean face; the cascaded regressor recovers part of the rotation through its iterative correction. (iii) Augmentation helps exactly where its distribution overlaps the perturbation: the augmented CNN is the best model at 40° rotation (0.230 vs. 0.243 for cascaded) and at scale 0.7 (0.170 vs. 0.181), but pays for that robustness with worse clean-data accuracy, a textbook bias–variance trade-off.

Generative AI declaration

Generative AI (ChatGPT and Claude) was used primarily during the exploratory phase of this project. Specifically, it helped me come up with ideas for the kinds of notebook cells worth writing in order to understand the data better, such as which descriptive statistics to compute, which slicing views of the corpus to plot, and which sanity checks to run on the face-alignment images before committing to a preprocessing pipeline. This shaped the structure of the early data-exploration notebooks and the choice of diagnostics that later fed into the spam-handling

and augmentation decisions.

Beyond that exploratory scaffolding, AI assistance was also used to speed up my understanding of some of the included Python libraries (for example clarifying the expected input shapes, parameter conventions and typical usage patterns of scikit-learn, gensim, scikit-image and PyTorch APIs), as well as for routine LaTeX formatting questions and for occasional debugging of Python errors. It was *not* used to write the prose of this report, to design the final modelling pipeline, or to analyse data end-to-end; all numerical results, figures and tables were produced by the author's own code in the accompanying notebooks, and all interpretation and discussion in this report are the author's own.

References

[1] Steven Bird, Ewan Klein, and Edward Loper. *Natural Language Processing with Python*. O'Reilly Media, 2009. `https://www.nltk.org`.

[2] Xudong Cao, Yichen Wei, Fang Wen, and Jian Sun. Face alignment by explicit shape regression. *International Journal of Computer Vision*, 107(2):177–190, 2014.

[3] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. Online manuscript, 3rd, draft edition, 2026. `https://web.stanford.edu/~jurafsky/slp3/`.

[4] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.

[5] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *International Conference on Learning Representations (ICLR), Workshop Track*, 2013.

[6] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[7] Bo Pang and Lillian Lee. A sentimental education: Sentiment analysis using subjectivity summarization based on minimum cuts. In *Proceedings of the 42nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 271–278, Barcelona, Spain, 2004.

[8] Martin F. Porter. An algorithm for suffix stripping. *Program*, 14(3):130–137, 1980.

[9] Xuehan Xiong and Fernando De la Torre. Supervised descent method and its applications to face alignment. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013.